

Ahmed Kiel-Kamil Data Structure Tutoring Guide

Tutoring is a very unique challenge as it is one of the first jobs open to us as students that requires both a high level of technical and social skills. Beyond just knowing the answer to everyone's problems you have to be able to explain it to them in a way that makes sense as well as provide students with useful practice to ensure they retain the information. Being able to help students at every level and with any kind of assistance is key. It is not enough to just be able to debug someone's code or answer someone's singular question without explaining the context of the subject matter. In this regard, the highest level of tutoring is helping someone go from being completely lost to possessing a workable understanding of the information. As a tutor, this should be your end goal of effectiveness, do not settle for just answering "on the fly" questions. The reason I am establishing this groundwork is because Dr. Ng's and Ali's Data Structures class are likely the hardest Computer Science classes due to their higher expectations. As tutors, our skills and abilities have to rise to an equally high standard to sufficiently support the students.

Part 1 (Soft Skills):

Soft skills are the "attributes that enable someone to interact effectively and harmoniously with other people". In the context of tutoring, you have to know how people need to be supported and make the experience as enjoyable as possible. The first step is creating a safe space. By the time students start coming to you for help they are likely out of options and very lost. On top of that, it is a common occurrence in Dr. Ngs and Ali's class for students to skip over tons of details if they are attempting to gather all the information too quickly. It is your job to ensure they feel comfortable to express what they don't know. In order to do you have to remove any hint of judgment as you correct them. They should know that you are the place they can go to for help even when they are entirely lost. Of course, this dynamic will change over time. As

you begin to see their improvement, tell them that. Especially, if they end up getting a problem wrong, make sure you point out the positives and the progress they make before correcting them. Motivation is a lot easier to maintain when you are told you are on the right track even if you're not at the finish line. On top of telling them that you see growth, recognize their growth by raising your expectations. The important distinction here is that, unlike unwanted pressure or judgement, your standard for them is rising based on what they have *shown* you. These positive expectations and affirmations ensure that the student knows that you believe in them which again goes into creating a safe space where they feel appreciated.

Part 2 (Knowledge Base):

Gathering the knowledge to effectively help students is easier said than done. In terms of what you need to do, you should at least attempt every function and data structure class from every set of notes besides ADT for Ngs class by yourself before we start next semester. On top of that, read the notes very carefully. A good benchmark is that you should feel comfortable explaining everything in these notes pre trees a Programming 2 student who has never heard of it. I know this may seem like a lot and it is. Going back to the intro, harder classes require better tutors. There is some light at the end of the tunnel though. Keep in mind 1: the more of his notes that you master the easier it is to understand how he explains concepts 2: mastering the concepts in theory will help you code the classes and coding the classes will help you master the concepts. Of course, I don't expect you to have everything done by the end of the winter, do your best with the core content and try to hit the stretch goals. Also, feel free to call me if you need any help. Remember this level of comprehension is the *end* goal and will allow you to perform the best as a tutor.

Part 3 (Teaching process):

Now let's combine soft skills with the expression of knowledge to explain how the teaching process works. As stated earlier the highest level of tutoring brings someone from 0-10% understanding to 90-100% understanding. This means you have to be prepared to provide value to people in every single range. The first thing you have to do is assess their level of understanding of the topic. You do this by asking them basic questions at first and ramping up the difficulty until you run into a concept they are unsure about. Based on that you can decide where their knowledge is and assist them accordingly.

0-20% comprehension: Explain the foundation concepts of the chapter in ways that are easily understandable. Then build from that foundation making sure the student understands how one smaller concepts links to the next. Eventually, have them complete a procedure on their own.

21-70% comprehension: Talk to them about the basics and ascend your conversation to the point where you're discussing procedures. If they are confident in the procedure, have them try it.

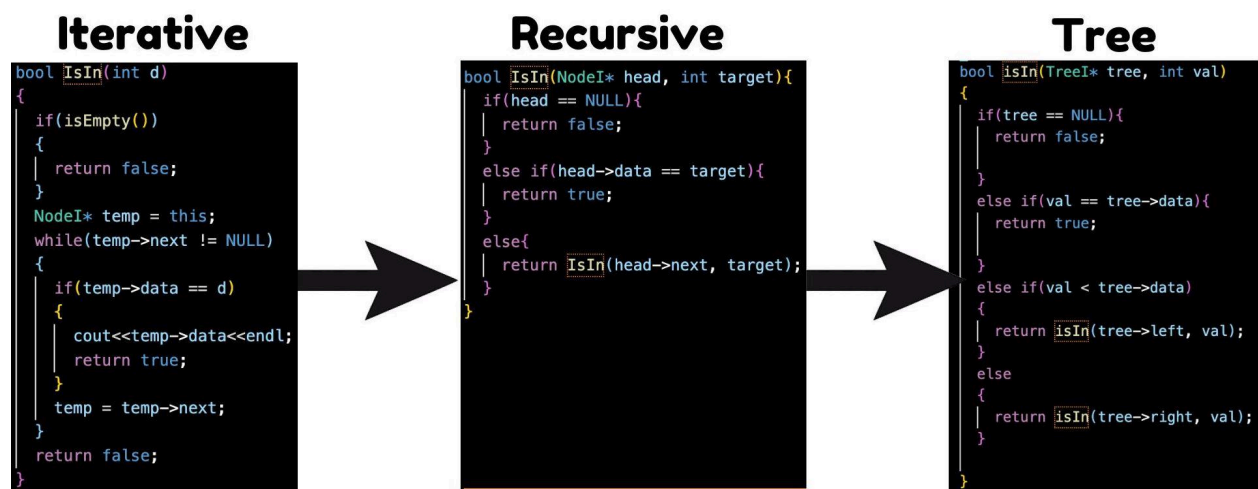
Oftentimes in this range, people have a theoretical understanding of the topic but their knowledge has holes that prevent them from fully completing the tasks that utilize these concepts. Your job is to find these gaps and fill them. Do this by asking them questions that lead them to the right answer. This improves their critical thinking skills and better prepares them to handle unexpected problems in the future. If through conversation you run into a portion where they get stuck, explain it using the existing knowledge that they already have.

71-90% comprehension: Have stretch goal questions prepared that force them to engage with all aspects of a certain chapter. Be more rigorous in your correction of both their definitions and programming, ensuring that they build code that accounts for unfavorable inputs (such as trying to find a number that is not in a list). It is important that you do this part in this section. Forcing a

student to engage with error checking too early will likely overwhelm them when they do not have the basics down. They may even end up prioritizing poorly, putting too much emphasis on error checking while neglecting their ability to create the algorithm properly to begin with. It is important to understand that while a student will still lose points for not error-checking, *nothing* is salvageable if they build a flawed algorithm.

A final note on the tutoring for Data Structures specifically. As mentioned earlier when approaching an new topic you want to start by slowly expanding upon what they already know. This really comes into play for the ladder half of the units (Trees, heaps, and hashmaps). These concept when taken in isolation can feel insurmountable so you have to explain in ways that feel like slight deviations of concepts they have already done before. Here is the lay out

Trees: Use their knowledge of link list to the fullest. First off, explain binary trees as a link list but instead of having 1 next node there are 2 that are denoted as left and right. When explaining recursion, have them write iterative link list functions first. Then convert it into a recursive function all the while showing them how the information carries over. Then start doing a basic tree function like is IsIn for a binary search tree.



Heaps: Use their knowledge of trees to the fullest. Start by explaining trees using arrays. Have them make is in for binary search trees using array implementation.

Hashmaps (chaining): use their knowledge of Link List and Arrays to the fullest: explain that these are just an array of link lists.

Part 4 (Programming assignments):

Challenges:

Compared to concepts and even standard homework assignments, long standing programming projects are a unique challenge to assist students with for 3 reasons. One, you can not as comfortably rely on your general computer science or programming knowledge to serve students. Due to the cause and effect nature of system design, the most obvious solutions built upon general knowledge are often suboptimal and can result in very tedious code that lacks modularity in the long run. Two, people's endpoints are different. Unlike a concept where everyone is looking to reach the same understanding, people will approach projects differently and therefore their end result will be different. This aspect of the process makes assisting with programs very time consuming because there is much less information that is universally valuable and each person will need individual assistance making the lecture approach null and void. The final issue is that of satisfaction. Unlike a concept where the student will be satisfied with full understanding and rudimentary application, a student will not be satisfied with their assistance on a project until they see progress made on their screen. This can prove challenging to facilitate if they either have an error that is well beyond their current understanding (ex. bus errors) or if a project requires a lot of set up and the visual output is at the very end. This issue also stifles your ability to educate them as not only will students be less interested in using tools unfamiliar to them that you can teach them about, but even the opportunities for a learning lesson

during the process will often be overshadowed by the status of their projects whether it's working or not. I am presenting you with these issues so that you may one, not make some of the mistakes mentioned such as expecting your general programming knowledge to serve them and 2 better illustrate how the solution handles these issues.

Solution steps:

1. In order to make sure you are able to best serve the students, if possible, get the assignments that the teacher will assign early. That way you have plenty of time to read the specification and come up with optimal system design.
2. **Do the assignment.** This is not optional. This is the best way to get an in-depth understanding of how the assignment should be built. That being said, you can not just complete this assignment like normal. Here are some factors you should consider when creating the assignment yourself:
 - a. Limit your use of tools that they have not learned about yet. The less truly new information you have to introduce, the easier it will be for them to retain the information you are giving them.
 - b. Try to find unique ways of utilizing the topics they are covering in class. This gives them the opportunity to cement their application of the concepts they are learning in class and pushes them to think outside of the box in terms of use cases.
 - c. Be cognizant of the inevitably tedious aspects of the program that may give students trouble.
 - d. If an aspect of your program is extremely complicated, find a way to simplify it

3. Build a mini curriculum for the assignment with different stages of progression. These modules should be general so that variance in approach can still be considered within the curriculum.
4. Practice teaching modules from the ground up and when there is flexibility on the order try to institute a linear learning curve as best you can avoiding big jumps in difficulty.
5. Get very familiar with the specifications and consider easily misunderstood instructions. You will likely have to explain these to them in more plain english.

Example:

Specification: Program Postfix Evaluation

Function

The program evaluates postfix arithmetic expressions containing real numbers and the operators +, -, *, and /.

Input

The input is a series of arithmetic expressions entered interactively from the keyboard in postfix notation. The user is prompted to enter an expression made up of operators (the characters '+', '-', '*', and '/') and real numbers. The end of the expression is marked by the expression terminator character, '='. Operators and operands must be terminated by at least one blank.

Real numbers must be expressed in decimal format: the whole part, followed by a decimal point ('.'), followed by the decimal part. The decimal point and the decimal part are optional. Exponential notation (that is, 3.2E3) is not valid.

The user terminates the program by entering the character "#" instead of a new expression.

Output

The user is prompted for each new expression to be evaluated with the following prompt:
"Enter expression to evaluate or a # to quit."

After the evaluation of each expression, the results are printed to the screen:

"Result = *value*"

where *value* is a real number in decimal format, with a field width of 8 and a decimal field of 4.

Processing Requirements

1. This program must read real-number inputs as characters and make the appropriate conversions.

Assumptions

1. The expressions are correctly entered in postfix notation.
2. Each expression is terminated by '='.
3. The operations in expressions are valid at run time. This means that we do not try to divide by zero.

Deliverables

- ◆ Program design and CRC cards for all classes
- ◆ A listing of your program including any classes used
- ◆ A listing of your test plan as input to the program
- ◆ A listing of the output file from the test runs

Postfix Assignment Curriculum

1. Concepts
 1. Explain postfix
 2. Explain stacks
 3. Explain where stacks come in for postfix
2. Break down system design
 1. Make a stack (Array or pointer based)
 2. Separate the postfix expression into a linked list parsed by spaces
 3. Make postfix functions
3. Make pointer stack
4. Make supporting tools
 1. Node struct and append function
 2. Substring function
 3. Type conversion
5. Make string to list to parse the string
6. Make the postfix functions (interactive or recursive)

When it comes to actually engaging with the students you will get 2 different types. Those with no existing approach who want to start from scratch with you. The benefit of working with these students is that you can take them through exactly how you did it and your curriculum set up will likely be much more effective. Additionally, you can likely foresee all the errors they might get and better explain the tools they are using. The disadvantage to this approach is that over time they may struggle to develop independent problem solving skills during the system design phase because they will be much more reliant on what you have already built for them. To account for this, present them with some of the smaller problems you ran into and push them to find their own methods of solving these problems. Over time, this will hopefully blossom into them being able to take on the full load of the planning stage with good ideas. The other kind of students you will get are those with pre-existing code. For these students you first have to take some time listening to their request and examining their code to see which part of the process they are currently on. If their problems are limited to code errors, show them how the tools they are work.

Then explain why their code does not work. Give them a chance to figure out how to solve the problem after you have offered the new perspective. If they are unable to figure it out, explain how to solve the issue in concept and let them code the solution. The obvious advantage to working with these kinds of students is that the amount of time spent fixing their code is usually much less because there is no preamble about algorithm planning. There are 2 disadvantages. One, due to their potentially under-developed system design skills, they may come up with an approach so suboptimal or tedious that you are better off wiping a section or the entirety of their code. In these situations be sure to first make sure that their current approach is truly not worth the hassle of actually being implemented, then be very respectful about your request to pivot their approach. If possible, point out the specifications that conflict with their approach to depersonalize your conclusion. The other problem is that they may get errors far beyond their skill level. In these situations, you likely do not have time to coach them towards fixing the error so just solve it for them and explain how you did it and have them explain it back to you.

Thank you for reading this guide. I know it is a lot to unpack, but I hope you are beginning to understand the kind of work necessary to properly serve these students. Our role is so important to the Morehouse Computer Science community. This is the point where a lot of students drop out or lose passion for tech. It is our job to show them data structures do not have to be the impossible hill that only peak-level coders can climb. Our job is to make this course accessible to those willing to show up and try. Therefore, we must go the extra mile to make sure we do this job well for the sake of the generations of black tech students coming up!